CDOClient

The CDOClient code is situated at OW2's git repository (https://tuleap.ow2.org/plugins/git/paasage/cdo_client). It can be used to connect to a CDOServer (https://tuleap.ow2.org/plugins/git/paasage/cdo_server) through a CDOSession and then make queries, store or export models / objects returned from queries.

Dependencies
The CDOClient depends on the library of CAMEL (https://tuleap.ow2.org/plugins/git/paasage/camel), which can also be found at the same git repository. This dependency has been included for the following reasons: (a) the main packages in the domain code of CAMEL are always registered once a CDOSession to a CDOServer is established and (b) the respective classes are needed in order to also support the transformation from xmi to textual CAMEL models and the opposite as well as loading textual CAMEL models.

Usage

Compilation
The CDOClient can be compiled through using maven and just executing the following command either at command line or your favourite Application Development framework: "mvn exec:java".

Execution
The CDOClient can be either included inside another component or can be executed in a standalone manner. In the second case, the following alternative execution ways are enabled (again either via a command line or your favourite Application Development framework):

- "mvn:exec java -Deu.paasage.configdir=<path_to_dir_properties_file>"
- "java -jar client-2.0.0-SNAPSHOT-jar-with-dependencies.jar" -Deu.paasage.configdir=<path_to_dir_properties_file>"

As you can see, in both cases you need to provide a directive to the java VM indicating where the properties file needed for the configuration of the CDOClient is situated in terms of a specific directory in the file system (by giving a full path to this directory). This directive is actually optional, in case you have set the PAASAGE_CONFIG_DIR environment variable with the full system path to the properties file directory. Of course, you can also provide input parameters to the main method, depending on the way you desire to exploit the CDOClient. The supply of these parameters can be performed in different ways depending on whether you perform this via the command line or an application development framework.

Exploitation
The CDOClient code can be exploited in various ways:

- *for testing purposes* to see that the main functionality exposed is correct. To this end, the main method includes a specific exploitation scenario which can be checked against a running CDOServer. This scenario involves the: (a) the programmatic creation of two models, one organisational model and one full use-case CAMEL model according to the SensApp use case as well as the loading of another use-case CAMEL model with respect to the Scalarm use case. The scenario also involves performing queries on the models stored as well as deleting some objects from one of these models. Finally, it includes the case of exporting a model as well as employing an event listener which outputs the events generated through the various model

management actions performed in this scenario.

- *for real exploitation*. In this case, we foresee that the CDOClient can be exploited inside one component which desires to perform particular model management actions or externally via command line e.g. in case that some model management actions are needed to be performed upon initialisation of the CDOServer.

For the external, command-line exploitation, the main method can take one or more input parameters which designate what method to call with which input. It should be noted that obviously in this case it is impossible to load in main memory objects/models and manipulate them as we are working via command line. Thus, the developer has to just call the desired methods and ensure that any model that may have to be used as input is properly identified. Having said that, the following patterns of exploitation should be followed via the command-line: <method_name> <arg1> <arg2> <arg3>. This means that at most 4 parameters should be provided where the first maps to the name of the method to call and the rest to the input parameters that should be used for calling this method. Each method takes 1 to at most 3 parameters so, in the end, the developer/user should provide at least 2 input parameters to the main method which map to the name of the method and its one input parameter.

We should highlight here that not all methods are supported in the pattern-based command-line execution of the CDOClient but only those related to the importing and exporting of models as well as to the XMI-to-CAMEL transformation. To this end, the main methods that are currently exploited are the following: *importModel*, *importTextualModel*, *xmiToCAMEL*, *exportModel*, *exportTextualModel*, *exportModelWithRef*, *exportModelWithRefRec*, *exportCDOContent* and *loadCDOContent*. Please note that the use of these models actually underlies the need of always importing models situated in files and not in the main memory as Java objects.

Main Features

The CDOClient exposes a particular functionality which enables the communication with a CDOServer and the issuing of model management actions on this server. The model management actions that are supported are the following:

1. Load a model from a XMI or textual (CAMEL) file in the main memory
2. Load a XMI or textual model and immediately store it in the CDOServer. The model to be stored can be first validated according to the OCL constraints that complement the semantics of EMF Ecore specified. In case it is valid, it is then stored. Otherwise, the method returns the value of false as output. It should be noted that irrespectively of whether the OCL constraint validation is on, the validation based on the EMF Ecore semantics is always performed before the model is stored. This guarantees that the model to be stored conforms to its meta-model(s) and only valid models are saved in the CDORepository.
3. Run a query on the CDOServer according to a specific query dialect (SQL and HQL are supported depending on the underlying CDORepository type of the CDOServer) and obtain back the results
4. Transform a XMI to a textual CAMEL model
5. Transform a textual CAMEL model to a XMI one
6. Export a model in a particular file location in the file system according to either the XMI or textual CAMEL format. This exporting can be performed in various alternative ways:
    1. export just the model - in case that it cross-references other models, these references will point to models in the CDORepository
    2. export the model as well as its cross-referenced models - in this case, the cross-references now map to files in the file system and this enables loading properly the exported model to any particular editor (such as the Eclipse EMF Tree-based Editor or the CAMEL textual editor). However, we should note that the additional models exported can still point to other models in the CDORepository

3. export the model as well as its cross-referenced models in a recursive manner in order to ensure that all exported models point to each other (thus staying only at the file system level) and not to any model stored in the CDORepository. In this way, all the exported models can then be viewed in any kind of editor.

The two latter exportation alternatives, apart from viewing purposes, they can also be exploited in case of component testing. For instance, imagine a chain of components which might be situated in a different machine (either virtual or physical) that have to be called in turn in order to produce a particular result (e.g., the components of the Upperware module in the PaaSage platform). If these components are not connected to a global CDOServer such that each can write its results to this server in order to make them available to the next components, then the testing could be performed in an individual manner when the locally produced result in the local CDOServer is exported and sent via other communication means by the developer of this component to the developer of the next component. As such, the next component can then read the exported models and perform its expected and realised functionality. This offline-based testing method was a feature request from the developers of the Upperware module which has been realised. In our opinion, provides an added-value to CDOClient and enables it to become a very powerful instrument for model-based code development and testing.

7. Export the whole content of the CDORepository in CAMEL textual or XMI form. This functionality is quite interesting in terms of back-up as it can be used in conjunction with the following method to restore the previous content of a CDORepository. The combinations of these methods could be exploited by an administrator of the PaaSage platform in order to back-up the content of the CDORepository, bring down the platform VM, do any kind of fixes, create a new platform VM and initialise the CDOServer/Repository in this VM with the content that has been exported. In this way, the models produced by the clients of the platform will not be lost when a repair or management action on the platform has to be performed. The exported content is in the form of a single zip file which encloses all the models of the CDORepository in textual CAMEL or XMI form.
8. Import a zip file with a previous content of a CDORepository to restore it. As indicated above, this completes the realisation of the back-up functionality.
9. Delete an object or model from the CDORepository in case it is no longer needed.

Apart from the model management actions, additional functionality has been realised to cover the following cases:

1. The developer needs to perform a sophisticated and composite model management action in a single transaction. In this case, the CDOClient offers the developer with the functionality of opening transactions (CDOTransaction) which can then be exploited to create resources, insert models, delete models or objects and so on. The developer, however, should be responsible for committing the transaction in the end and closing it. The developer may also need to do some model processing without making any update in the CDORepository. In this case, the CDOClient offers to the developers the functionality of opening views (CDOView) to enable them to make queries and process the objects returned from them. It should be highlighted here that in case of manipulation of models which are stored and loaded from the CDORepository, there should always be a transaction or view open through which the model has been loaded. Otherwise, if an attempt to manipulate a CDO-based model is performed out of the context of any transaction or view, an exception will be raised.
2. The developer needs to manipulate non-CAMEL models. In this case, the CDOClient offers the functionality of registering the packages of the meta-models on which these models conform in the CDORepository in order to guarantee that the management actions performed on them will be error-free.

3. The developer needs to develop a component which should be aware of particular events that happen inside a CDORepository in terms of, e.g., the storage or update of particular types of models. For instance, the Adapter component of the Upperware might be interested to check when new high-level measurements for a metric are stored in the CDORepository in order to check whether adaptation actions have to be performed on the application at hand. As such, the CDOClient offers a particular method in order to register a MyListener object on the current CDOSesssion which emits all events created in the context of model manipulations in the respective CDORepository.

The aforementioned functionality of the CDOClient can be exploited in order to support some advanced model manipulation scenarios. Such scenarios can take the following form:

1. Make a query to the CDOServer in order to find and load particular information and then either change this information and store it back to CDOServer, or create a model out of it and then store it in the CDOServer or just export the query result. This more complicated scenario can be useful for the following cases:
   (a) the user desires to modify a model. Thus, he/she has first to perform a query in order to obtain it (or can directly obtain it from a CDOView or CDOTransaction via its CDOID) and then modify it and store it back to the CDOServer. Imagine, for instance, the case of a feature model of a particular provider that has to be modified when a new cloud service is offered by this provider;
   (b) the user desires to create a model out of an existing model or a part of it. Thus, he/she has to issue a query to fetch it and then browse the results in order to create the corresponding model to be stored in the CDOServer. Imagine, for instance, the case where we have to associate the CERIF (eurocris.org/**cerif**/main-features-**cerif**) information from a cloud provider with the feature model that this provider offers. When the cerif information is created, we will have to obtain the feature model of the cloud provider and then make the appropriate association between the instance of CloudProvider and the instance of the FeatureModel (see property providerModel in the organisation subpackage of CAMEL meta-model);
   (c) the user wants to view a particular output, which is not a complete model but maps to parts or objects of this model, from the PaaSage platform. In this case, a particular component code will have to issue the respective query needed and then export the result as an XMI or CAMEL textual file. Finally, the XMI/textual file produced can then exploited by an editor, such as the Eclipse EMF Tree-based editor or the CAMEL textual editor in order to produce the respective model-based visualisation.
2. Make a query or directly fetch a particular object to be removed from the CDOServer. This scenario can include the repetition of the above retrieval and deletion actions many times until all desired objects are removed. Please be aware that the respective methods used to realize the scenario should be used with care as they do not anticipate the following situation: when the object is deleted from its containment or non-containment association, the respective object to which it is associated may no longer be valid. For instance, if the object to be deleted is associated with a one-to-one relation with its associating object, then by removing it the associating object becomes invalid. As another example, if there is an one-to-many association which should not be empty, then if the deleted object is the last one in the association, this will make its associating object invalid. These problematic situations, if not properly addressed, will certainly lead to exceptions when objects have to be deleted. It is thus advocated that the developer should always follow these constraints as well as attempt to perform a set of deletions always in the context of a single transaction as otherwise, if some of the deletions are

performed in an isolated manner and leave the respective model in an invalid state, then exceptions will be raised.

3. Export models that are suitable for visualisation via editors. In this scenario, the developer/user has just to call the respective method of the CDOClient which hides the complexity involved in exporting models as well as fixing their references to point only to file-based models. The CDOClient offers two methods to support this scenario: (a) *exportModelWithRef* and (b) *exportModelWithRefRec* depending on whether the visualisation should focus on only the basic model or also the models that it cross-references.
4. Business continuity scenario: In this scenario, the content of the CDORepository is backed up via calling the *exportCDOContent* method of the CDOClient either at regular time points or when maintenance/bug fixing tasks have to be performed on the running PaaSage platform instance. After the instance execution is resumed or a new version of it is being deployed, the content that has been backed up can be restored via calling the CDOClient's *loadCDOContent* method. The calling of these two methods can be either performed in a programmatic manner or via scripts which call the main method of the CDOClient with the appropriate input parameters mapping to calling one of these two back-up/restore methods.
5. Event-based model manipulation action scenario: In this scenario, the code of the developer listens on model manipulation events on the CDORepository in order to act upon the manipulation of objects or models of a particular type (e.g., to create another model or modify the existing one by adding extra information to it). To support this scenario, the developer has to create his own IListener/MyListener class extension in order to check which objects from those updated/added/deleted are of interest such that the respective model manipulation code can be executed. A particular instance of this scenario in the context of the PaaSage platform has already been identified previously.

Method Detailed Analysis

The CDOClient code comprises just one main class called "CDOClient" which can be exploited to realize the above exploitation scenarios. This class comprises the following methods:

- <u>*Name*</u>: **registerPackage**
  - *Description*: This method is used for registering an EPackage that maps to the domain code to be manipulated. We take the view that the user is responsible for registering any EPackage that does not correspond to those mapping to the CAMEL and WP3 meta-models, as the system will not be aware of a new meta-model and the respective domain code.
  - *Input Parameters*:
    - **pack** : The *EPackage* object to be registered. (***Obligatory***)
  - *Output*: No output needs to be produced
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the user desires to register the CP meta-model package of WP3 (Upperware), then the following code will have to be produced: cl.registerPackage(CpPackage.eINSTANCE);
- <u>*Name*</u>: **openTransaction**
  - *Description*: This method is used to open a CDO transaction and return it to the user. Then, the user can issue queries or store models through the direct exploitation of this transaction (object) instead of using the corresponding methods exposed by the CDOClient class. It

must be noted that once all user tasks are ended, the CDO transaction created needs to be closed by calling the closeTransaction method with the transaction object as input parameter.

- *Input Parameters*:
  - **validate** : Boolean parameter indicating whether models to be stored/updated should conform to the OCL constraints posed. (***Optional***)
- *Output*: An object of class *CDOTransaction*
- *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the user can obtain the CDO transaction created as follows: CDOTransaction trans = cl.openTransaction();

- *Name*: **openView**
  - *Description*: This method is used to open a CDO view and return it to the user. Then, the user can issue queries through the direct exploitation of this view (object) instead of using the corresponding methods exposed by the CDOClient class. It must be noted that once all corresponding user tasks are ended, the CDO view created needs to be closed by calling the closeView method with the view object as input parameter.
  - *Input Parameters*:
    - None
  - *Output*: An object of class *CDOView*
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the user can obtain the CDO view created as follows: CDOView view = cl.openView();
- *Name*: **closeTransaction**
  - *Description*: This method is used to close a previously opened CDO transaction, obviously after all corresponding user tasks have been executed.
  - *Input Parameters*:
    - **trans** : The *CDOTransaction* object to be closed.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and a *CDOTransaction trans* has been opened, the user can close the CDO transaction created as follows: CDOTransaction trans = cl.closeTransaction(trans);
- *Name*: **closeView**
  - *Description*: This method is used to close a previously opened CDO view, obviously after all corresponding user tasks have been executed.
  - *Input Parameters*:

        - **view** : The *CDOView* object to be closed.
    - *Output*: None
    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and a *CDOView view* has been opened, the user can close the CDO view created as follows: CDOView view = cl.closeView(view);
- *Name*: **storeModel**
    - *Description*: This method is used to store a model in the CDO Server under a CDOResource with a particular name. Please note that no transaction needs to be created for performing this storage as this is conducted internally by the method.
    - *Input Parameters*:
        - **model** : The model to be stored as an instance of *EObject* class (which is indeed the case as all models created are instances of classes that are subclasses of EObject).
        - **resourceName** : The name of the CDOResource as a String under which the model will be stored.
        - **validate** : A boolean parameter indicating whether the model should be validated against the OCL constraints (posed in its meta-model) before it is stored.
    - *Output*: None
    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and a *EObject model* has been created (programmatically or via calling the loadModel method of CDOClient class), the user can store this model under a CDOResource named as "camel1" as follows: cl.storeModel(model,"camel1",true);
- *Name*: **loadModel**
    - *Description*: This method is used to load a model from a XMI file in the file system. Upon loading, then the model can be read, processed and possibly stored in the CDO Server.
    - *Input Parameters*:
        - **pathName** : The path (as a String) to the XMI file containing the resource/model to load
    - *Output*: The model as instance of EObject class
    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the file containing the model to be loaded is named as "model.xmi" situated in the current directory "input" sub-directory, then the user can load the desired model as follows: EObject model = cl.loadModel("input/model.xmi");
- *Name*: **loadTextualModel**
    - *Description*: This method is used to load a model from a CAMEL textual file in the file system. Upon loading, then the model can be read, processed and possibly stored in the

CDO Server.

- *Input Parameters*:
  - **pathName** : The path (as a String) to the CAMEL textual file containing the resource/model to load
- *Output*: The model as instance of EObject class
- *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the file containing the model to be loaded is named as "model.camel" situated in the current directory "input" sub-directory, then the user can load the desired model as follows: EObject model = cl.loadTextualModel("input/model.camel");

- *Name*: **loadModel** (overloaded)
  - *Description*: This method is used to load a model from a XMI file given its URL. Upon loading, then the model can be read, processed and possibly stored in the CDO Server.
  - *Input Parameters*:
    - **url** : The URL from which the XMI file can be downloaded and loaded into main memory.
  - *Output*: The model as instance of EObject class
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the file containing the model to be loaded is located at "www.mymodels.com/model.xmi", then the user can load the desired model as follows: EObject model = cl.loadModel(new URL("www.mymodels.com/model.xmi"));
- *Name*: **loadTextualModel** (overloaded)
  - *Description*: This method is used to load a model from a CAMEL textual file given its URL. Upon loading, then the model can be read, processed and possibly stored in the CDO Server.
  - *Input Parameters*:
    - **url** : The URL from which the CAMEL textual file can be downloaded and loaded into main memory.
  - *Output*: The model as instance of EObject class
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the file containing the model to be loaded is located at "www.mymodels.com/model.camel", then the user can load the desired model as follows: EObject model = cl.loadTextualModel(new URL("www.mymodels.com/model.camel"));
- *Name*: **loadModel** (overloaded)
  - *Description*: This method is used to load a model from a XMI file given its respective *InputStream*. Upon loading, then the model can be read, processed and possibly stored in the

CDO Server.

- *Input Parameters*:
  - **is** : The InputStream from which the XMI file can be loaded into main memory.
- *Output*: The model as instance of EObject class
- *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the file containing the model to be loaded is named as "model.xmi" and is located at the "input" sud-directory of the current directory, then the user can load the desired model as follows: EObject model = cl.loadModel(new FileInputStream("input/model.xmi"));

- *Name*: **importModel**
  - *Description*: This method is used to load a model from a XMI file and immediately store it in the CDO Server.
  - *Input Parameters*:
    - **filePath** : The path (as a String) to the XMI file containing the resource/model to be imported
    - **resourcePath** : The path (as a String) on the CDORepository on which to create the respective resource to hold the model to be imported
    - **validate** : Boolean parameter indicating whether the model should be validated against the OCL constraints before it is stored in the CDORepository
  - *Output*: A boolean value indicating whether the importing has been successful or not.
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the file containing the model to be loaded is named as "model.xmi" situated in the current directory's "input" sub-directory and that the resource path in the CDORepository on which the model is to be stored is "orgs/forth/kritikos@ics.forth.gr/kritikos@ics.forth.gr" (see also model storage guidelines) [D4.1.2], then the user can import the desired model as follows: cl.importModel("input/model.xmi","orgs/forth/kritikos@ics.forth.gr/kritikos@ics.forth.gr",true);

- *Name*: **importTextualModel**
  - *Description*: This method is used to load a model from a CAMEL textual file and immediately store it in the CDO Server.
  - *Input Parameters*:
    - **filePath** : The path (as a String) to the CAMEL textual file containing the resource/model to be imported
    - **resourcePath** : The path (as a String) on the CDORepository on which to create the respective resource to hold the model to be imported

        - **validate** : Boolean parameter indicating whether the model should be validated against the OCL constraints before it is stored in the CDORepository

    - *Output*: A boolean value indicating whether the importing has been successful or not.

    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the file containing the model to be loaded is named as "model.camel" situated in the current directory's "input" sub-directory and that the resource path in the CDORepository on which the model is to be stored is "orgs/forth/kritikos@ics.forth.gr/kritikos@ics.forth.gr" (see also model storage guidelines) [], then the user can import the desired model as follows: cl.importTextualModel("input/model.camel","orgs/forth/kritikos@ics.forth.gr/kritikos@ics.forth.gr",true);

- *Name*: **loadCDOContent**

    - *Description*: This method is used to load a zip file containing a set of models into the CDORepository. This file can map to the content of a whole repository which has been previously backed up. It can contain file entries which all are either in CAMEL textual or XMI form.

    - *Input Parameters*:

        - **filePath** : The path (as a String) to the zip file containing the content to be loaded in the CDORepository

        - **xtext** : Boolean parameter indicating whether the models mapping to file entries in the zip file are either of the CAMEL textual (true) or XMI (false) form.

    - *Output*: A boolean value indicating whether the content loading has been successful or not.

    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the zip file is situated at the relative "input" sub-directory, named as "cdo.zip", and contains file entries in CAMEL textual form, then the user can load the zip file's content as follows:

      cl.loadCDOContent("input/cdo.zip", true);

- *Name*: **createCerifModel**

    - *Description*: This method is used to create (programmatically) a cerif model (instance of OrganisationModel class) which can then be stored in the CDO Server by calling the storeModel method. This method has been realised for testing and showcasing purposes.

    - *Input Parameters*:

        - None

    - *Output*: The model as an instance of EObject

    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, you can create and obtain the cerif model as follows: EObject model = cl.createCerifModel();

- *Name*: **exportModel** (overloaded with two method instances - the **first** is described here)

- *Description*: This method is used to export a model that has been stored under a particular CDOResource into a XMI file with a particular path. We should highlight that if the required model is not placed at the root of the CDOResource designated by the user (which actually maps to a CamelModel), then we consider the first model belonging to the CamelModel of the same type as the one requested by the user to be exported (e.g, the first DeploymentModel belonging to the CamelModel referenced by exploiting the containment association *deploymentModels*). If a different model of the same type is desired, then it is better to exploit the second instance of the method (analysed below) after the desired model has been obtained through a query. We must also note that the user is responsible for providing correct input parameters as well as ensuring that the requested model has been stored under the designated CDOResource.
- *Input Parameters*:
  - **resourceName** : The name (as a String) of the CDOResource under which the model has been stored.
  - **c** : The class (java *Class* object) which is instantiated by the required model to be stored (e.g., DeploymentModel.class).
  - **filePath** : The path to the file (as a String) on which the required model will be stored.
- *Output*: None
- *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the file to be used for storing the model has the relative path "output/model.xmi", the name of the CDOResource is "camel1" and a model of type DeploymentModel needs to be stored, then the exporting will be performed as follows:

```
cl.exportModel("camel1",DeploymentModel.class,"output/model.xmi");
```

- *Name*: **exportModel** (overloaded with two method instances - the **second** is described here)
  - *Description*: This method is used to export a model, which is provided directly as input parameter, into a XMI file with a particular name. The model could be created programmatically or retrieved by issuing a query. We must highlight here that the method should be called after a CDOTransaction/CDOView has been created, as otherwise an Exception will be thrown (this is a restriction from CDO which requires that a transaction must have been opened in order to be able to manipulate a model from the CDORepository).
  - *Input Parameters*:
    - **model** : The model to be stored as instance of *EObject* class.
    - **filePath** : The path to the file (as a String) on which the required model will be stored.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the model to be exported is named as *model* and maps to a DeploymentModel in CAMEL, and the file to be used for storing the model has the relative path "output/model.xmi", then the exporting will

be performed as follows: cl.exportModel(model, "output/model.xmi");

- *Name*: **exportTextualModel**
  - *Description*: This method is used to export a model, which is provided directly as input parameter, into a CAMEL textual file with a particular name. The model could be created programmatically or retrieved by issuing a query. We must highlight here that the method should be called after a CDOTransaction/CDOView has been created, as otherwise an Exception will be thrown (this is a restriction from CDO which requires that a transaction must have been opened in order to be able to manipulate a model from the CDORepository).
  - *Input Parameters*:
    - **model** : The model to be stored as instance of *EObject* class.
    - **filePath** : The path to the file (as a String) on which the required model will be stored.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the model to be exported is named as *model* and maps to a DeploymentModel in CAMEL, and the file to be used for storing the model has the relative path "output/model.camel", then the exporting will be performed as follows: cl.exportTextualModel(model, "output/model.camel");
- *Name*: **exportModelWithRef**
  - *Description*: This method is used to export a model into the file system along with the models that it cross-references. In this way, the exported model points only to file system based models and this facilitates its proper visualisation via a model-based editor. The cross-referenced models can point, however, to other models which are CDO-based, i.e., they are stored in the CDORepository. The model could be created programmatically or retrieved by issuing a query. We must highlight here that the method should be called after a CDOTransaction/CDOView has been created, as otherwise an Exception will be thrown (this is a restriction from CDO which requires that a transaction must have been opened in order to be able to manipulate a model from the CDORepository).
  - *Input Parameters*:
    - **resourcePath** : The path as a String to the CDORepository pointing to the resource in which the model has been stored.
    - **dirPath** : The path as a String to the file system directory which will contain the exported models
    - **fileName**: The name of the file which will contain the primary model exported
  - *Output*: A boolean value indicating whether the exporting has been successful
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the model to be exported is contained in a *CDOResource* at the following path "orgs/forth/kritikos@ics.forth.gr/kritikos@ics.forth.gr", the name of the file to store this model is named as "kritikos.xmi" and the path to the directory which will contain this file as

well as the exported cross-referenced models as files is "output", then the exporting will be performed as follows:

```
cl.exportModelWithRef("org/forths/kritikos@ics.forth.gr/kritikos@ics.forth.gr",
"output", "kritikos.xmi");
```

- *Name*: **exportModelWithRefRec**
  - *Description*: This method is used to export a model into the file system along with the models that it cross-references in a recursive manner such that all exported models point only to file system-based models. As such, all the exported models are amenable to proper visualisation via a model-based editor. The model could be created programmatically or retrieved by issuing a query. We must highlight here that the method should be called after a CDOTransaction/CDOView has been created, as otherwise an Exception will be thrown (this is a restriction from CDO which requires that a transaction must have been opened in order to be able to manipulate a model from the CDORepository).
  - *Input Parameters*:
    - **resourcePath** : The path as a String to the CDORepository pointing to the resource in which the model has been stored.
    - **dirPath** : The path as a String to the file system directory which will contain the exported models
    - **xtext**: A boolean parameter indicating whether the models will be exported in a CAMEL textual or XMI form.
  - *Output*: A boolean value indicating whether the exporting has been successful
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the model to be exported is contained in a *CDOResource* at the following path "orgs/forth/kritikos@ics.forth.gr/kritikos@ics.forth.gr", and the path to the directory which will contain the exported files is "output", then the recursive exporting in CAMEL textual form will be performed as follows:

    ```
    cl.exportModelWithRefRec("org/forths/kritikos@ics.forth.gr/kritikos@ics.forth.gr",
    "output", true);
    ```

- *Name*: **exportCDOContent**
  - *Description*: This method is used to export the whole content of a CDORepository in either CAMEL textual or XMI form in a zip file (thus, each model is mapped to a CAMEL textual or XMI file entry which is included in this zip file) named as "cdo.zip". This method is appropriate for back up reasons and it is complemented by the loadCDOContent method that was detailed above.
  - *Input Parameters*:
    - **dirPath** : The path as a String to the file system directory which will contain the zip file to be created

        - **xtext**: A boolean parameter indicating whether the models will be exported in a CAMEL textual or XMI form.

    - *Output*: A boolean value indicating whether the massive CDO content exporting has been successful

    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and the path to the directory which will contain the exported files is "output", then the CDORepository content exporting in CAMEL textual form will be performed as follows:

      cl.exportCDOContent("output", true);

- *Name*: **runQuery**

    - *Description*: This method is used to run a query over a CDO Server / Repository and obtain back a list of instances of EObject class. Three types of query dialects are currently supported: OCL, SQL and HQL. The use of a dialect depends on the underlying repository type of the CDOServer. OCL is always supported but HQL is only supported by a HibernateStore. Through this method, the user has also the option to export the first object from the list to be returned into a XMI file.

    - *Input Parameters*:

        - **dialect** : The dialect of the query as a String with three possible values: "OCL", "SQL" and "HQL".

        - **queryStr** : The actual query as a String which obviously should conform to the dialect given.

        - **fileName** : The name of the file (as a String) on which the first instance of *EObject* class (i.e., first query result) will be stored. The file will be situated under the directory designated by the PAASAGE_CONFIG_DIR system variable.

    - *Output*: None

    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the query dialect is HQL, the desired query is "`select dm from DeploymentModel dm`" and the file to be used for storing the first query result is named as "queryResult.xmi", then the query issuing will be performed as follows: List<EObject> results = cl.runQuery("HQL","select dm from DeploymentModel dm", "queryResult.xmi");

- *Name*: **xmiToCAMEL (static)**

    - *Description*: This method is used to transform a file-based XMI model to a file-based textual CAMEL model. This can be useful in the case that a user has exported in the past a XMI file and desires to transform it into CAMEL textual form in order to be able to visualise and edit it via the CAMEL's textual editor.

    - *Input Parameters*:

        - **xmiPath** : The path in the file system where the XMI model is located

        - **camelPath** : The path in the file system where the CAMEL textual model is to be created

    - *Output*: A boolean value indicating whether the transformation was successful or not

    - *Usage*: By supposing that the XMI file is located at "input/model.xmi" and that the CAMEL textual model has to be generated at "output/model.camel", then the xmi-to-CAMEL transformation can be performed as follows:

      CDOClient.xmiToCAMEL("input/model.xmi","output/model.camel");

- *Name*: **deleteObject (**overloaded with two method instances - the **first** is described here**)**

    - *Description*: This method is used to delete an object from the CDO Server / Repository based on its CDOID provided as input.

    - *Input Parameters*:

        - **cdoID** : The CDOID of the object to be deleted.

    - *Output*: None

    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that a query has been issued to obtain the object *obj* to be deleted, then the object deletion can be performed as follows: cl.deleteObject(obj.cdoID());

- *Name*: **deleteObject (**overloaded with two method instances - the **second** is described here**)**

    - *Description*: This method is used to delete an object from the CDO Server / Repository by also exploiting the transaction that is given as input.

    - *Input Parameters*:

        - **object** : The CDOObject to be deleted.

        - **trans** : The CDOTransaction under which the deletion will occur.

        - **commitAndClose** : A boolean parameter indicating whether the transaction given as input will be committed and closed or not. In the latter case, the user is responsible for either committing and closing the transaction him/herself or providing the last call of this method with the value of true for this parameter.

    - *Output*: None

    - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that a query has been issued via transaction *trans* to obtain the object *obj* to be deleted, then the object deletion which should be immediately committed can be performed as follows: cl.deleteObject(obj, trans, true);

- *Name*: **closeSession**

    - *Description*: This method is used to close the CDOSession that is opened when a

CDOClient object is created. Thus, this method should be the very last one from those that will be exploited for a particular user task / exploitation scenario.

  - *Input Parameters*: None
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and used/exploited, the very last command to be executed will be as follows: cl.closeSession();

- *Name*: **addListener**
  - *Description*: This method is used to add a listener (instance of IListener) to the current CDOSession in order to listen to the model manipulation actions performed on the respective CDORepository. A specific class extending IListener was developed in the CDOClient code called MyListener which just prints the objects that have been added, deleted or modified. This class can be either extended by the developers or they can create their own extension classes to IListener.
  - *Input Parameters*:
    - **listener** : The IListener object to be added.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created as well as a MyListerer object *listener* has been generated, the addition of this listener object can be performed as follows: cl.addListener(listener);

- *Name*: **removeListener**
  - *Description*: This method is used to remove a listener in order not to listen any more to events generated via model manipulation actions on the respective CDORepository.
  - *Input Parameters*:
    - **listener** : The IListener object to be removed.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created as well as a MyListerer object *listener* has been generated and already registered, the removal/deregistration of this listener object can be performed as follows: cl.removeListener(listener);

Sample Code Fragments Mapping to Exploitation Scenarios

Now, we are going to show code fragments that realize the five exploitation scenarios aforementioned for complicated model management.

Exploitation Scenario 1 - Retrieve objects and process them:

```
//Create CDOClient object
CDOClient cl = new CDOClient();

//Open a  CDOTransaction
Transaction trans = cl.openTransaction();

//Obtain all objects under a specific CDOResource named as "camel1"
List<EObject> results = trans.getResource("camel1").getContents();

//Process each object
for (EObject obj: results){
System.out.println("Got object: " + obj);
//do sth with the object
......
}

//Close the opened transaction
cl.closeTransaction(trans);

//Close CDO Session
cl.closeSession();
```

Exploitation Scenario 2 - Retrieve objects and delete them in two alternative ways:

```
//Create CDOClient object
CDOClient cl = new CDOClient();

//1. Beginning of first deletion alternative
//Open a  CDOTransaction
Transaction trans = cl.openTransaction();

//Obtain object to be deleted
DataCenter dc = trans.createQuery("hql","select dc from DataCenter dc where
dc.codeName="AWS-europe"").getResult(DataCenter.class).get(0);

//Delete Object as well as commit and close transaction
cl.deleteObject(dc, trans, true);

//2. Beginning of second deletion alternative
//Open a CDO view
CDOView view = cl.openView();

//Get object to be deleted
Role role = view.createQuery("hql","select role from Role role where
role.name="MyRole"").getResult(Role.class).get(0);

//Delete Object
```

```
cl.deleteObject(role.cdoID());

//Close CDO View
cl.closeView(view);

//Close CDO Session
cl.closeSession();
```

Exploitation Scenario 3 - Model export for visualisation:

```
//Create CDOClient object
CDOClient cl = new CDOClient();

//Export model and its cross-referenced models recursively in CAMEL textual form
cl.exportModelWithRefRec("orgs/forth/kritikos@ics.forth.gr/kritikos@ics.forth.gr","output",true
);

//Close session
cl.closeSession();

...

launch CAMEL textual editor to see the models exported
```

Exploitation Scenario 4 - back up and restore CDORepository content:

```
//Create CDOClient object
CDOClient cl = new CDOClient();

//Export whole content of CDO Repository in CAMEL textual form into the back-up directory.
cl.exportCDOContent("back-up", true);

//Close CDOSession
cl.closeSession();

... do sth like a maintenance or debugging task & re-launch PaaSage platform

//Create CDOClient object
CDOClient cl = new CDOClient();

//Load content of CDO Repository in CAMEL textual form previously backed up.
cl.loadCDOContent("back-up/cdo.zip", true);

//Close CDOSession
cl.closeSession();
```

Exploitation Scenario 5 - listen to model manipulation events in CDORepository and act upon them:

Implement the IListener interface and focus on the types of object you desire to process (*DeploymentModel*s in this case):

```
public class MyListener implements IListener{
      private CDOTransaction trans;

      public MyListener(CDOTransaction trans){
            this.trans = trans;
      }

      //This method is called when an event is fired by the CDOSession
      public void notifyEvent(IEvent arg0) {
            if (arg0 instanceof CDOSessionInvalidationEvent){
                  CDOSessionInvalidationEvent e =
(CDOSessionInvalidationEvent)arg0;
                  List<CDOIDAndVersion> objs = e.getNewObjects();
                  for (CDOIDAndVersion id: objs){
                        //New Objects
                        CDOObject obj = trans.getObject(id);
                        if (obj instanceof DeploymentModel){
                            do sth ....
                        }
                  }
                  List<CDORevisionKey> modObjs = e.getChangedObjects();
                  for (CDORevisionKey id: modObjs){
                        //Updated Objects
                        CDOObject obj = trans.getObject(id);
                        if (obj instanceof DeploymentModel){
                            do sth ....
                        }
                  }
            }
      }

}


//Create CDOClient object
CDOClient cl = new CDOClient();

//Create the MyListener object and register it
CDOTransaction trans = cl.openTransaction();
MyListener listener = new MyListener(trans);
cl.addListener(listener);

... do other things ...

//Deregister MyListener object when done
cl.removeListener(listener);

//Close CDOSession
```

```
cl.closeSession();
```

Support

For any inquiries about the CDOClient code, please contact Kyriakos Kritikos via the following email address: kritikos AT ics.forth.gr.

References

[D4.1.2] D. A. Imrie, A. Innes, T. Kirkham, Korozi, M., Kritikos, K., Kryza, B., Leonidis, A., Magoutis, K., Massonet, P., Ntoa, S., Papaioannou, A., Papoulas, C., Sheridan, C.: D4.1.2 – Product Database and Social Network System (September 2015).